developerWorks®

Anatomia do Kernel Linux

Histórico e Decomposição Arquitetural

M. Tim Jones
Consultant Engineer
Emulex Corp.

06/Jun/2007

O kernel Linux® é o núcleo de um sistema operacional complexo e grande e, apesar do tamanho, é bem organizado em termos de subsistemas e camadas. Neste artigo, explore a estrutura geral do kernel Linux e conheça seus principais subsistemas e interfaces centrais. Sempre que possível, haverá links para outros artigos da IBM que o ajudarão a se aprofundar no assunto.

Como o objetivo deste artigo é apresentá-lo o kernel Linux e explorar sua arquitetura e componentes principais, vamos começar por um breve tour pelo histórico do kernel Linux, em seguida, observaremos sua arquitetura e, finalmente, examinaremos seus principais subsistemas. O kernel Linux é formado por mais de seis milhões de linhas de código, portanto, esta introdução não é completa. Para informações adicionais, utilize os ponteiros.

Um Breve Tour pelo Histórico do Linux

Linux ou GNU/Linux?

Provavelmente foi notado que o Linux, como sistema operacional, às vezes é referido como "Linux" e, outras vezes, como "GNU/Linux." O motivo é que o Linux é o kernel de um sistema operacional. A ampla diversidade de aplicativos que tornam um sistema operacional útil forma o software GNU. Por exemplo, o sistema com janelas, o compilador, a variedade de shells, as ferramentas de desenvolvimento, os editores, os utilitários e outros aplicativos existem externamente ao kernel, muitos dos quais são softwares GNU. Por isso, muitos consideram "GNU/Linux" um nome mais apropriado para o sistema operacional, enquanto "Linux" é adequado para se referir somente ao kernel.

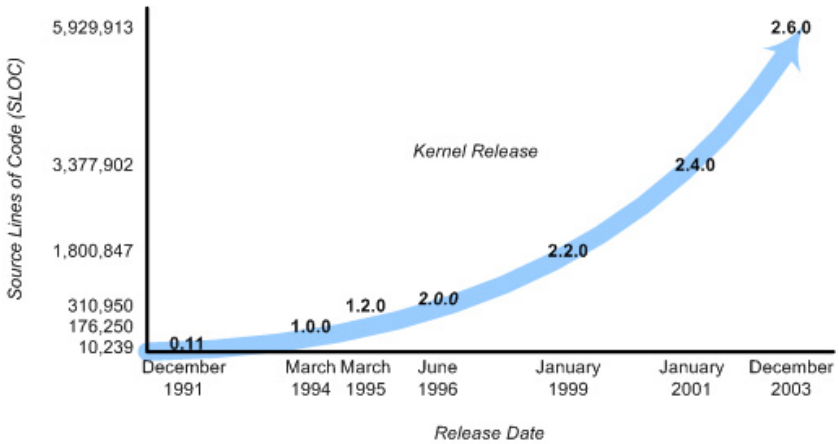
Embora o Linux seja possivelmente o sistema operacional de software livre mais popular, seu histórico é, na verdade, bastante breve, considerando a cronologia dos sistemas operacionais. Nos primórdios da computação, os programadores desenvolviam diretamente sobre o hardware, na linguagem dele. A ausência de um sistema operacional significava que apenas um aplicativo (e um usuário) por vez poderia utilizar o dispositivo grande e dispendioso. Os primeiros sistemas operacionais foram desenvolvidos nos anos 1950, para fornecer uma experiência mais simples de desenvolvimento. Como exemplos temos o Sistema Operacional da General Motors (GMOS), desenvolvido para o IBM 701, e o FORTRAN Monitor System (FMS) desenvolvido pela North American Aviation para o IBM 709.

© Copyright IBM Corporation 2007. Todos os direitos reservados.

Nos anos 1960, o Instituto de Tecnologia de Massachusetts (MIT) e um grupo de empresas desenvolveram um sistema operacional experimental chamado Multics (ou Multiplexed Information and Computing Service) para o GE-645. Um dos desenvolvedores desse sistema operacional, o AT&T, retirou-se da Multics e desenvolveu seu próprio sistema operacional em 1970, chamado Unics. Junto a esse sistema operacional havia a linguagem C, para a qual C foi desenvolvido e, em seguida, reformulado para tornar portátil o desenvolvimento do sistema operacional.

Vinte anos depois, Andrew Tanenbaum criou uma versão de microkernel do UNIX®, chamada MINIX (ou minimal UNIX), que era executada em computadores pessoais. Esse sistema operacional de software livre inspirou o desenvolvimento inicial de Linus Torvalds da Linux no começo dos anos 1990 (consulte a Figura 1).

Figura 1. Breve Histórico dos Principais Releases do Kernel Linux

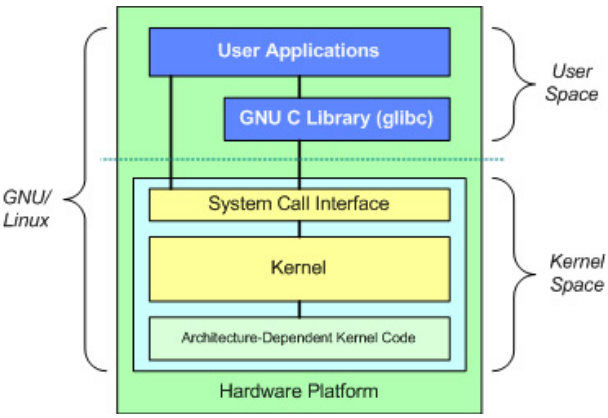


O Linux evoluiu rapidamente de um projeto de uma só pessoa para um projeto mundial de desenvolvimento envolvendo milhares de desenvolvedores. Uma das mais importantes decisões para o Linux foi adotar a GNU General Public License (GPL). Sob a GPL, o kernel Linux ficava protegido da exploração comercial e também se beneficiava do desenvolvimento do espaço de usuário do projeto GNU (de Richard Stallman, cujo código de origem impede o do kernel Linux). Isso possibilitou aplicativos úteis, como o GNU Compiler Collection (GCC), e vários suportes de shell.

Introdução ao Kernel Linux

Agora passemos a uma visão mais ampla da arquitetura do sistema operacional GNU/Linux. É possível imaginar um sistema operacional a partir de dois níveis, conforme mostra a Figura 2.

Figura 2. A Arquitetura Fundamental do Sistema Operacional GNU/Linux



Métodos para a Interface de Chamada do Sistema (SCI)

Na verdade, a arquitetura não é tão clara quanto mostrada na Figura 2. Por exemplo, o mecanismo pelo qual as chamadas do sistema são manipuladas (mudando do espaço de usuário ao espaço do kernel) pode ter uma diferença de arquitetura. As Unidades de Processamento Centrais (CPUs) x86 mais novas que oferecem suporte para instruções de virtualização são mais eficazes neste processo do que os processadores x86 mais antigos, que usam o método tradicional int 80h.

Na parte superior, temos o espaço de usuário ou do aplicativo. Este é o espaço no qual os aplicativos de usuário são executados. Abaixo do espaço de usuário está o espaço do kernel. Aqui, o kernel Linux existe.

Há também a GNU C Library (glibc). Ela fornece a interface de chamada do sistema que se conecta ao kernel e fornece o mecanismo para transição entre o aplicativo de espaço de usuário e o kernel. Isso é importante, pois o kernel e o aplicativo do usuário ocupam espaços de endereços diferentes e protegidos. E embora cada processo de espaço de usuário ocupe seu próprio espaço de endereço virtual, o kernel ocupa um único espaço de endereço. Para obter informações adicionais, consulte os links na seção Recursos.

O kernel Linux pode ainda ser dividido em três níveis completos. Na parte superior, a interface de chamada do sistema, que implementa as funções básicas, como read e write. Abaixo da interface de chamada do sistema está o código do kernel, que pode ser mais precisamente definido como o código do kernel independente da arquitetura. Esse código é comum a todas as arquiteturas do processador às quais o Linux oferece suporte. Abaixo, está o código dependente da arquitetura, que forma o que é mais comumente chamado de BSP (Board Support Package). Este código atua como o processador e o código específico da plataforma para a arquitetura em questão.

Propriedades do Kernel Linux

Ao discutir a arquitetura de um sistema amplo e complexo, é possível visualizar o sistema sob várias perspectivas. Um objetivo de uma decomposição de arquitetura é fornecer um modo de compreender melhor a origem, e é isso que faremos aqui.

O kernel Linux implementa vários atributos importantes de arquitetura. Em um nível avançado e em níveis mais baixos, o kernel é dividido em camadas em diversos subsistemas distintos. O Linux pode também ser considerado monolítico porque agrupa todos os serviços básicos dentro do kernel. Isso o diferencia da arquitetura de microkernel, na qual o kernel fornece serviços básicos, como comunicação, E/S e

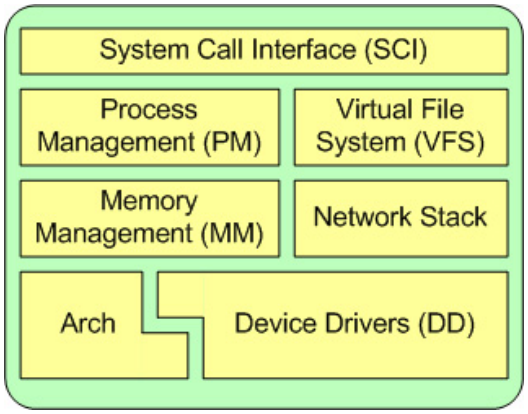
gerenciamento de memória e processo, e serviços mais específicos são plugados na camada do microkernel. Cada um tem suas próprias vantagens, mas vou me afastar desse debate.

Com o tempo, o kernel Linux tornou-se eficaz tanto em termos de memória quanto de uso da CPU, além de ser extremamente estável. Contudo, o aspecto mais interessante do Linux, considerando seu tamanho e sua complexidade, é sua portabilidade. Ele pode ser compilado para executar uma grande quantidade de processadores e plataformas com diferentes limitações e necessidades de arquitetura. Um exemplo é a capacidade do Linux em executar em um processo com uma Memory Management Unit (MMU), bem como em processos que não fornecem MMU. A porta uClinux do kernel Linux é fornecida para suporte a não-MMU. Consulte a seção Recursos para obter mais detalhes.

Principais Subsistemas do Kernel Linux

Vejamos agora alguns dos principais componentes do kernel Linux utilizando a perspectiva mostrada na Figura 3 como guia.

Figura 3. Uma Perspectiva de Arquitetura do Kernel Linux



Interface de Chamada do Sistema

A SCI é uma camada fina que fornece meios para efetuar as chamadas de funções a partir do espaço de usuário no kernel. Conforme discutido anteriormente, essa interface pode ser dependente da arquitetura, mesmo dentro da mesma família de processadores. A SCI é, na verdade, um interessante serviço de multiplexação e desmultiplexação de chamada de funções. É possível encontrar a implementação da SCI em ./linux/kernel, bem como as partes dependentes da arquitetura em ./linux/arch. Mais detalhes desse componente estão disponíveis na seção Recursos.

Gerenciamento de Processos

O que É um Kernel?

Como mostrado na Figura 3, um kernel, na verdade, não é nada mais do que um gerenciador de recursos. Se o recurso que está sendo gerenciado for um processo, uma memória ou um dispositivo de hardware, o kernel gerencia e intermedeia o acesso ao recurso entre os vários usuários concorrentes (no kernel e no espaço do usuário).

O gerenciamento de processo tem foco na execução de processos. No kernel, eles são chamados de encadeamentos e representam uma virtualização individual do processador (código de encadeamento, dados, pilha e registros de CPU). No espaço do usuário, o termo processo geralmente é utilizado, embora

a implementação do Linux não separe os dois conceitos (processos e encadeamentos). O kernel fornece uma Interface de Programação de Aplicativo (API) através da SCI para criar um novo processo (funções fork, exec, ou Portable Operating System Interface [POSIX]), parar um processo (kill, exit) e executar a comunicação e sincronização entre eles (mecanismos signal ou POSIX).

Além disso, no gerenciamento de processo há a necessidade de compartilhar a CPU entre os encadeamentos ativos. O kernel implementa um novo algoritmo de planejamento que opera em tempo constante, independentemente do número de encadeamentos competindo pela CPU. Isso é chamado de planejador O(1), denotando que se leva a mesma quantidade de tempo para planejar um encadeamento ou muitos. O planejador O(1) também oferece suporte a vários processadores (chamado Symmetric MultiProcessing, ou SMP). É possível encontrar as origens do gerenciamento de processo em ./linux/kernel e as origens dependentes de arquitetura em ./linux/arch). É possível saber mais sobre esse algoritmo na seção Recursos.

Gerenciamento de Memória

Outro importante recurso gerenciado pelo kernel é a memória. Em termos de eficiência, dada a forma como o hardware gerencia a memória virtual, a memória é gerenciada no que são chamadas páginas (de 4KB na maioria das arquiteturas). O Linux inclui os meios para gerenciar a memória disponível, assim como os mecanismos de hardware para mapeamento físico e virtual.

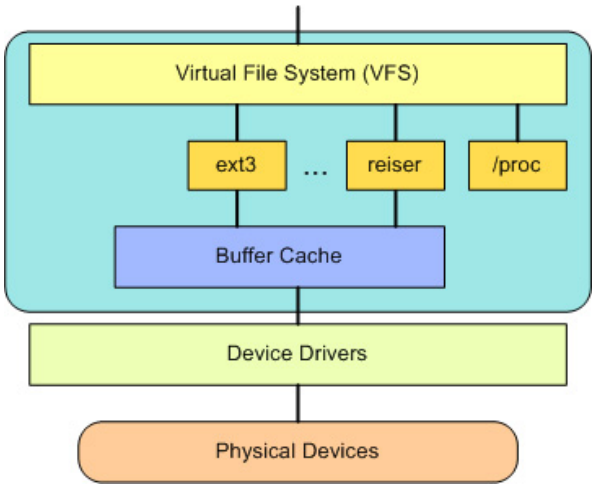
Mas o gerenciamento de memória é muito mais do que gerenciar buffers de 4KB. O Linux fornece abstrações acima de buffers de 4KB, como o alocador slab. Esse esquema de gerenciamento de memória utiliza buffers de 4KB como base, mas, em seguida, aloca estruturas internamente, rastreando quais páginas estão completas, parcialmente usadas e vazias. Isso permite que o esquema aumente e diminua dinamicamente, com base nas necessidades do sistema geral.

Ao oferecer suporte de memória a vários usuários, há ocasiões em que a memória disponível pode esgotar-se. Por isso, as páginas podem ser movidas da memória para o disco. Esse processo é chamado de troca porque as páginas são trocadas da memória para o disco rígido. É possível encontrar as origens de gerenciamento de memória em ./linux/mm.

Sistema de Arquivo Virtual

O Sistema de Arquivo Virtual (VFS) é um aspecto interessante do kernel Linux, porque fornece uma abstração de interface aos sistemas de arquivos. O VFS fornece uma camada de troca entre a SCI e os sistemas de arquivos aos quais o kernel oferece suporte (consulte a Figura 4).

Figura 4. O VFS Fornece uma Malha de Comutação Entre Usuários e Sistemas de Arquivos



No topo do VFS há uma abstração comum de funções da API como open, close, read e write. Na parte inferior do VFS encontram-se as abstrações do sistema de arquivo que definem como as funções da camada superior são implementadas. Elas são plug-ins para o sistema de arquivos fornecido (existem mais de 50 deles). É possível encontrar as origens do sistema de arquivos em ./linux/fs.

Abaixo da camada do sistema de arquivos está o cache do buffer, que fornece um conjunto comum de funções à camada do sistema de arquivos (independente de qualquer sistema de arquivos específico). Essa camada de armazenamento em cache otimiza o acesso aos dispositivos físicos mantendo os dados por perto durante um breve período (ou especulativamente já lidos, para que fiquem disponíveis quando necessários). Abaixo do cache do buffer estão os drivers de dispositivo, que implementam a interface para o dispositivo físico específico.

Pilha de Redes

A pilha de redes, pela estrutura, segue uma arquitetura em camadas modelada após os próprios protocolos. Lembre-se de que o Protocolo de Internet (IP) é o protocolo principal de camadas de rede situado abaixo do protocolo de transporte (mais comumente o Protocolo de Controle de Transmissões ou TCP). Acima do TCP está a camada de soquetes, que é chamada pelo SCI.

A camada de soquetes é a API padrão para o subsistema de rede e fornece uma interface com o usuário para vários protocolos de rede. Desde o acesso a quadros brutos às unidades de dados de protocolo IP (PDUs) e até o TCP e o User Datagram Protocol (UDP), a camada de soquetes fornece um modo padronizado de gerenciar conexões e mover dados entre terminais. É possível encontrar as origens de rede no kernel em ./linux/net.

Drivers de Dispositivo

A maioria dos códigos-fonte no kernel Linux existe nos drivers de dispositivo que tornam utilizável um determinado dispositivo de hardware. A árvore de códigos-fonte do Linux fornece um subdiretório de drivers que é posteriormente dividido pelos vários dispositivos suportados, como Bluetooth, I2C, serial, etc. É possível encontrar as origens do driver de dispositivo em ./linux/drivers.

Código Dependente de Arquitetura

Embora grande parte do Linux seja independente da arquitetura na qual ele é executado, existem elementos que devem considerar a arquitetura para o funcionamento normal e por questões de eficácia. O subdiretório ./linux/arch define a parte dependente da arquitetura da origem do kernel contido em diversos subdiretórios específicos a esta arquitetura (formando coletivamente o BSP). Em um desktop típico, é utilizado o diretório i386. Cada subdiretório de arquitetura contém diversos outros subdiretórios que se concentram em um aspecto particular do kernel, como boot, kernel, gerenciamento de memória e outros. É possível encontrar o código dependente da arquitetura em ./linux/arch.

Recursos Interessantes do Kernel Linux

Se a portabilidade e a eficácia do kernel Linux não bastarem, ele fornece outros recursos que não puderam ser classificados na decomposição anterior.

Sendo o Linux um sistema operacional em produção e um software livre, ele é um grande teste para novos protocolos e aprimoramentos desses protocolos. O Linux oferece suporte a um grande número de protocolos de rede, inclusive o típico TCP/IP, e também uma extensão para rede de alta velocidade (acima de 1 Gigabit Ethernet [GbE] e 10 GbE). O Linux também oferece suporte a protocolos como o Stream Control Transmission Protocol (SCTP), que fornece vários recursos avançados, acima do TCP (como o protocolo de nível de transporte de substituição).

O Linux também é um kernel dinâmico, oferecendo suporte à inclusão e remoção de componentes de software durante a execução. Eles são chamados módulos de kernel dinamicamente carregáveis, e podem ser inseridos durante a inicialização quando necessários (quando um determinado dispositivo é encontrado, solicitando o módulo) ou a qualquer momento pelo usuário.

Um avanço recente no Linux é seu uso como sistema operacional para outros sistemas operacionais (chamado de hypervisor). Recentemente, foi feita uma modificação no kernel, chamada Kernel-based Virtual Machine (KVM). Essa modificação permitiu uma nova interface para o espaço do usuário, possibilitando que outros sistemas operacionais fossem executados no kernel ativado pelo KVM. Além de executar outra instância do Linux, o Microsoft® Windows® também pode ser virtualizado. A única restrição é que o processador subjacente deve oferecer suporte às novas instruções de virtualização. Consulte a seção Recursos para obter informações adicionais.

Indo Além

Este artigo apenas pincelou a superfície da arquitetura do kernel Linux, bem como suas características e recursos. É possível verificar o diretório de Documentação fornecido em cada distribuição Linux para obter informações detalhadas sobre o conteúdo do kernel. Certifique-se de verificar a seção Recursos no final deste artigo para obter informações mais detalhadas sobre muitos dos tópicos discutidos aqui.

Sobre o autor

M. Tim Jones

M. Tim Jones é um arquiteto de firmwares embarcados e autor de Inteligência Artificial: Sistemas de Abordagem, GNU/Linux, Programação de Aplicativos AI (atualmente em sua segunda edição), Programação de Aplicativos AI (em sua segunda edição) e BSD Sockets Programming from a Multilanguage Perspective. Sua formação em engenharia vai desde o desenvolvimento de kernels para nave espacial geossincrônica até a arquitetura de sistema embarcado e o desenvolvimento de protocolos de interligação de redes. Tim é um Engenheiro Consultor para a Emulex Corp. em Longmont, Colorado.

© Copyright IBM Corporation 2007. Todos os direitos reservados.
(www.ibm.com/legal/copytrade.shtml)
Marcas Registradas
(www.ibm.com/developerworks/br/ibm/trademarks/)